Mini: architektura a vlastní projekt

Cesta od zdrojového programu k běhu; bloky 02–04

Karel Šafr

Bloková výuka

Finální Mini a jeho záměr

Mini je malý jazyk, na kterém lze sledovat celé zpracování programu.

- Funkce, lokální proměnné, bloky, `if`, `for`, `return` a `print`.
- Čtyři druhy hodnot: číslo, text, pravdivostní hodnota a `nil` (absence jiné hodnoty). Typy se kontrolují při běhu operací.
- Překládá se do C++17; hotový program využívá malou běhovou knihovnu.

Doma doplňujeme C++ implementaci překladače; programy Mini slouží jako vstupy testů. Potřebujeme základy C++17: třídy, reference a práci se stromem.

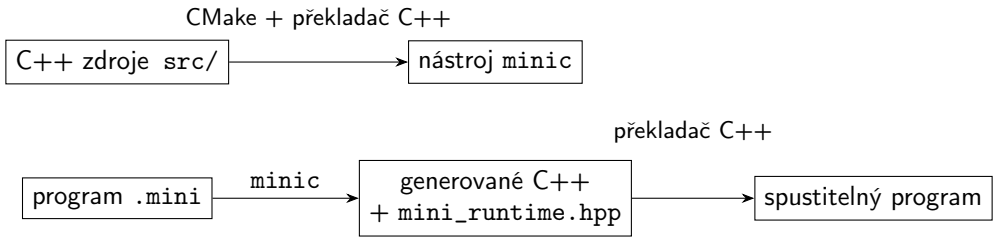
Současná verze je textbfMini 1.0.

Finální ukázka: kam společně směřujeme

```
fun twice(x) { return x * 2; }
fun main() {
  var total = 0;
  for (var i = 1; i <= 3; i = i + 1) {
    total = total + twice(i);
  }
  print total;
}
```

- `fun` zavede funkci, `var` proměnnou; `main` je vstup programu.
- `twice` vrací dvojnásobek argumentu. Sečteme $2 + 4 + 6$.
- Výstup finále: 12. Start bloku 02 ještě nemá obecné funkce ani smyčky.

Dvakrát C++: nástroj a výsledný program



- První sestavení vytváří náš překladač Mini.
- Druhé sestavení vytváří program napsaný studentem v Mini.
- CMake popisuje sestavení; C++ překladač provede překlad.

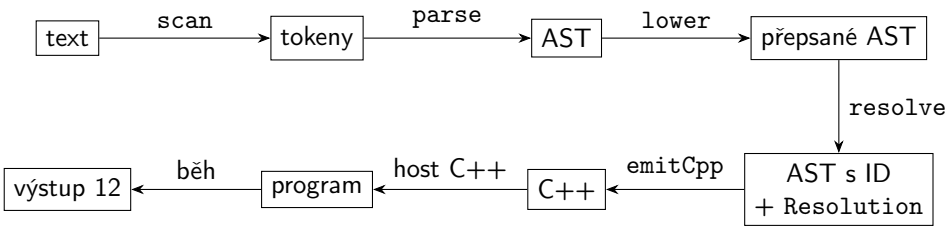
Kde hledat odpovědnost v projektu

Cesty jsou relativní ke kopii finálního `Mini_2_0`.

Soubor / skupina	Odpovědnost
`src/main.cpp, cli.*, driver.*`	Příkaz, vstup a řízení celé cesty.
`src/source.*, token.*, lexer.*`	Text, pozice, tokeny.
`src/ast.*, parser.*, lower.*`	Strom programu a jeho přepis.
`src/resolver.*, codegen.*`	Vazby jmen a vytvoření C++.
`src/host_compiler.*, diagnostics.*`	Spuštění C++ nástrojů a hlášení chyb.
`runtime/mini_runtime.hpp`	Hodnoty a operace vykonávaného programu.

`.hpp` obvykle zveřejňuje rozhraní, `.cpp` obsahuje implementaci; `.*` zde znamená oba soubory.

Jak program prochází překladačem



- **AST** je abstraktní syntaktický strom: struktura programu.
- Každá šipka má jiný úkol. Resolver nic nesčítá; parser nespouští funkce.
- Runtime je připojen ke generovanému programu; jeho operace se provedou až při běhu.

Driver vybírá, kde se cesta zastaví

Každý příkaz znovu načte zdroj: `./build/minic PŘÍKAZ demo.mini`.

Příkaz	Co získáme
`tokens`	Tokeny po lexeru.
`ast`	Strom po parseru; ještě obsahuje `ForStatement`.
`lower`	Strom po přepisu `for`.
`check`	Úspěšnou kontrolu jmen a volání; bez běhu.
`cpp`	Generovaný zdroj C++.
`build` / `run`	Sestavený program / sestavení a okamžité spuštění.

`cli.cpp` rozpozná požadavek; `driver.cpp` propojí fáze; `host_compiler.cpp` zavolá externí C++ překladač.

Lexer: text a původ tokenů

```
return x * 2;
RETURN  IDENTIFIER(x)  STAR  NUMBER(2)  SEMICOLON
```

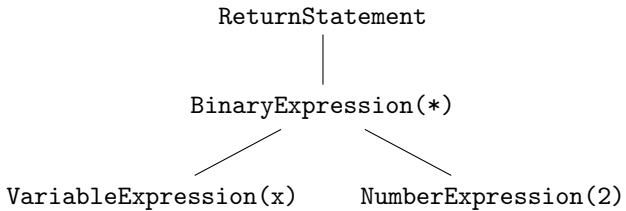
Token je rozpoznaná jednotka zápisu: klíčové slovo, jméno, číslo či operátor. Druhý řádek je jejich zkrácený přehled.

- `SourceText` uchová soubor a text; `scan` v `lexer.cpp` vytvoří tokeny.
- `Token` nese druh, text či hodnotu a `SourceSpan`: pozici úseku ve zdroji.
- Pozice umožní později ukázat, kde je chyba. Lexer ještě neví, ke které deklaraci patří `x`.

Nepovolený znak `@` se odmítne už zde.

Parser: z tokenů vznikne strom

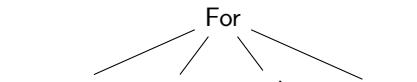
`parser.cpp`: funkce `parse` sestaví `Program`; druhy uzlů definuje `ast.hpp`.

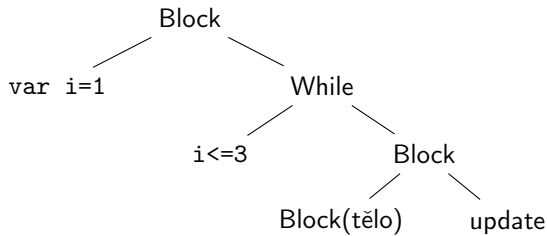


- Strom pro `return x * 2;` říká, co je operand a co se vrací.
- `Program` obsahuje funkce; jejich těla obsahují příkazy.
- Uzel výrazu používá `std::variant`: jednu z dovolených podob. Rekurzivní hrany výrazů vlastní `unique_ptr`; seznamy uzlů uchovává `vector`.

Lowering: složitější zápis na jednodušší uzly

`lower.cpp` přepíše `ForStatement`; tělo ukázky sčítá `twice(i)`.

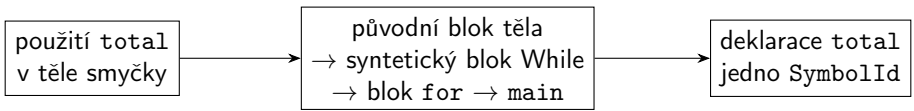




- Vnější blok vymezuje, kde lze použít jméno `i`; původní blok těla zůstává zachován.
- Test předchází každé iteraci; update je krok `i = i + 1` po těle.
- `while` je interní uzel, **není** zdrojové klíčové slovo Mini.

Resolver: jméno získá identitu deklarace

`resolver.cpp`: **rozsah** je oblast dostupných deklarací. Při vstupu do bloku přidáme novou oblast; hledáme od nejbližší ven.



- `resolved_id` v AST propojí čtení nebo zápis s deklarací.
- Funkce mají `FunctionId`; resolver ověří jejich existenci a počet argumentů (aritu).
- `Resolution` vrátí ID vstupní funkce `main`; hodnotu `total=12` nepočítá.

Generátor C++: příkazy a jména

`codegen.cpp`: `emitCpp` vytvoří text; `FunctionEmitter` zpracuje těla funkcí.

Mini / AST	Výsledný postup v C++
`var total = 0;`	Lokální `mini::Value`; jméno odvozené od `SymbolId`.
`print total;`	Volání `mini::printValue(...)`.
`return x * 2;`	Výpočet výrazu a C++ `return`.
Interní `WhileStatement`	Smyčka, která znovu počítá podmínku a kontroluje bool.

Dvě stejnojmenné proměnné dostanou různá jména, například `mini_variable_1` a `mini_variable_2`.

Generátor musí zachovat pořadí výpočtu

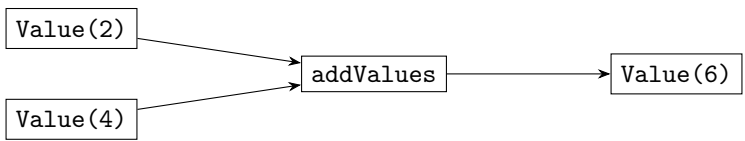
Mini stanoví: levý operand před pravým; argumenty volání zleva doprava. Výřez generovaných kroků pro `x * 2`:

```
const mini::Value mini_temp_0 = mini_variable_0;
const mini::Value mini_temp_1 = mini::Value{2.0};
const mini::Value mini_temp_2 =
    mini::multiplyValues(mini_temp_0, mini_temp_1);
return mini_temp_2;
```

- `mini_temp` uloží mezivýsledek; pomocná funkce běhové knihovny dostane hotové hodnoty.
- U `and`/`or` generátor vloží pravý operand do podmíněné větve. Zda se provede, rozhodne až běh.

Runtime: hodnoty a pravidla operací

Runtime je běhová knihovna pro přeložený program. Soubor `mini_runtime.hpp` poskytuje hodnoty, operace, tisk a kontroly.



- `Value` drží jednu alternativu: `double`, `bool`, `string` nebo prázdnou `monostate` pro `nil`.
- `2 + 4` dá 6, `"a" + "b"` dá `"ab"`; `2 + "b"` je chyba.
- `requireBool` kontroluje podmínky; `divideValues` odmítne dělení nulou.

Chyba ukazuje porušené pravidlo

Každý řádek je samostatná varianta těla `fun main() { ... }`.

Ukázka	Fáze	Co se kontroluje
`print @;`	lexer	Nepřípustný znak.
`print 1`	parser	Chybí středník před koncem bloku.
`print absent;`	resolver	Jméno nemá deklaraci.
`print 1 + "a";`	runtime	Nepřípustné druhy operandů.
`print 1 / 0;`	runtime	Pravý operand je nula.

Poslední dvě varianty projdou `check`, ale selžou při `run`. Chyba externího C++ nástroje je ještě jiný, backendový problém.

Sestavení a ověření finální reference

Ve vlastní kopii `Code/Mini/Mini_2_0`; přiložené `demo.mini` nejprve zkopírujeme do její složky `ukazky/`.

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build
./build/minic run ukazky/demo.mini
```

- Pracujeme ve vlastní kopii; finální referenci neupravujeme.
- Sestavení ověří použitelnost C++ kódu. Test navíc porovná výstup, chybovou fázi a návratový kód s očekáváním.
- V domácích balíčcích: `python3 overit.py -prekladac build/minic`. Na nevyřešený start přidáme `-vychozi`.

Zelený test režimu `-vychozi` potvrzuje očekávané TODO, nikoli vyřešenou úlohu.

Výuka a tři domácí kroky

Téma výuky	Co vidíme v projektu	Domácí celek
Zápis, jména, rozsahy; kap. 4–6	Lexer, parser, AST, resolver; druhy hodnot	Blok 02: celý `Resolver`.
Výrazy, řízení, funkce; kap. 7–9	Pořadí výpočtu, přepis `for`, generování C++	Blok 03: `emitStatement`.
Pravidla operací a hranice jazyka	Běhové kontroly a porovnání návrhových voleb	Blok 04: `applyBinary`.

- Každý blok začíná **novým připraveným startem** s potřebnou infrastrukturou.
- Přenášíme porozumění; staré celé soubory automaticky nekopírujeme.
- Mini ilustruje část teorie. Další paradigmata a abstrakce studujeme i mimo jeho možnosti.

Blok 02: celý resolver

Pracujeme doma ve vlastní kopii `TeachingBlocks/block02-start`. Upravujeme pouze celou C++ třídu `Resolver` v `src/resolver.cpp`.

- Start: `tokens/ast/lower` fungují; `check` skončí `TODO BLOCK02`.
- Úkol: oblasti jmen, ID, nejbližší deklarace, duplicity a vlastní inicializátor.
- Solution: bloky, proměnné, přiřazení a tisk v jediném `main`.

```
fun main() {
  var x = 10;
  { var x = 20; print x; }
  print x;
}
```

Výstup řešení: 20, pak 10.

Ještě bez operátorů, smyček a obecných funkcí. Umím vysvětlit, proč se obě `x` liší a co jejich ID neříká o hodnotě?

Blok 02: vstup, výsledek a hranice úkolu

Kontrakt: `ZADANI.md`; uzly: `src/ast.hpp`. Zachováme rozhraní:

```
Resolver(const SourceText &source, Program &program);
Resolution run();
```

Součást startu	Co znamená pro naši práci
`Program.main_body`	Příkazy `main` v pořadí zdroje; bez obecných funkcí.
`resolved_id`	Doplníme ID deklarací, čtení a cílů přiřazení přímo do AST.
`Binding{id, defined}`	ID a příznak kontroly inicializátoru; žádná běhová hodnota.
`Resolution{}`	Prázdný návratový objekt; tento blok nemá `FunctionId`.

`run`: otevřít rozsah → projít `main_body` → uzavřít → `Resolution{}`.

Blok 02: deklarace a přiřazení nejsou totéž

Každou variantu vložíme zvlášť do `main` za `var x = 10;`; po jejím bloku následuje ještě `print x;`.

Vnitřní blok	Vazba a pozorovaný výsledek
`{ var x = 20; print x; }`	Nové ID pro vnitřní `x`. Výstup `20, 10`.
`{ x = 20; print x; }`	Stále ID vnějšího `x`. Výstup `20, 20`.
`{ var x = x; }`	Nová, ještě nedefinovaná vazba; chyba resolveru. Vnější `x` se nepoužije.

Deklarace probíhá ve třech krocích:

1. Zapsat nové jedinečné ID a `defined=false` do aktuálního rozsahu.
2. Zkontrolovat inicializátor, tedy výraz pro počáteční hodnotu.
3. Teprve potom nastavit `defined=true`.

ID nerecyklujeme po odchodu z bloku. Hledáme od nejbližšího rozsahu ven; nedefinovanou vazbu nesmíme přeskočit.

Blok 02: od prvního sestavení k odevzdání

Ve vlastní kopii `Code/Mini/TeachingBlocks/block02-start`.
Nástroje: CMake 3.20+, C++17; pro ověřovač Python 3.10+.

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build --config Debug
python3 overit.py --prekladac build/minic --vychozi
```

Start: dostupné fáze a očekávané TODO. Po doplnění C++ třídy:

```
cmake --build build --config Debug
python3 overit.py --prekladac build/minic
./build/minic run priklady/01_shadowing.mini
```

- Po **novém sestavení** projde deset příkladů. Balíčkový `01_shadowing.mini` vypíše 20, pak 30.
- Odevzdat `src/resolver.cpp`, kladný i záporný vlastní příklad s očekávaným i skutečným výsledkem a vysvětlení chyby `var x=x`.

Blok 03: generování příkazů

Doplnit `FunctionEmitter::emitStatement` v `src/codegen.cpp`.

- Start: `check` platného programu projde; `cpp/run` skončí `TODO BLOCK03`.
- Dodány jsou parser, lowering, resolver i generování výrazů.
- Úkol: příkazy, bloky, větvení, návrat a opakovaný test smyčky.

```
fun main() {
  for (var i = 0;
       i < 3; i = i + 1) {
    print i;
  }
}
```

Výstup řešení: 0, 1, 2.

Hotová etapa umí funkce, volání, rekurzi, `if`/`for`; ještě bez `/` a spojování textů přes `+`. Proč musí test zůstat uvnitř smyčky?

Blok 04: operace a dosažení finále

Doplnit `applyBinary` v `runtime/mini_runtime.hpp`.

- Start se sestaví a kontroluje jména; až provedená binární operace vyvolá `TODO BLOCK04`.
- Úkol: všech deset aritmetických a porovnávacích operací, dovolené druhy a chyby.
- `and`/`or` zůstávají v generátoru kvůli zkrácenému vyhodnocení.

```
fun main() {
  print 5 / 2;
  print "Mini " + "jazyk";
  print true == 1;
}
```

Řešení: `2.5`, `Mini jazyk`, `false`.

Solution odpovídá chování finálního Mini. Runtime je kvůli úloze jinak uspořádaný; shodné chování neznamená totožné soubory.

Návrh pro kurz: dvě projektové cesty

	A: implementace Mini	B: teoretický návrh jazyka
Hlavní práce	Doplnit tři předepsané celky C++.	Navrhnout přesná pravidla vlastního malého jazyka.
Důkaz výsledku	Kód, testy, výstupy a jejich vysvětlení.	Gramatika, pravidla, stromy a ručně provedené výpočty.
Společný cíl	Vysvětlit vazby, význam operací, řízení toku a chyby na novém vstupu.	

A navíc prokazuje implementační dovednost; B ji nenahrazuje.

V obou případech je nutné dodat materiály k prezetnaci a obhajobě (postup řešení všech týdnů / prezetnace / teoretický návrh) 7 dní před obhajobou.

Teoretická cesta: co konkrétně odevzdat

Rozsah: **8–12 stran specifikace** a příloha s příklady.

1. Účel, cílový uživatel, podporované konstrukce a dvě zdůvodněné návrhové volby; také co jazyk neumí.
2. Tokeny a úplná malá gramatika EBNF (pravidla zápisu): literály, výrazy, deklarace, přiřazení, blok, podmínka, smyčka a výpis. Jasná priorita operátorů.
3. Pravidla hodnot, rozsahů, pořadí výpočtu a chyb. Použité symboly a stavy vysvětlit.
4. Nejméně deset případů: šest platných a čtyři chybné (syntax, jméno, druh hodnoty, nepřípustná hodnota), vždy s očekávaným výsledkem a konkrétním pravidlem.
5. U tří platných programů tokeny, AST, vazby a ruční stopa stavu; jeden musí obsahovat smyčku.

Jde o proveditelnou definici jazyka. Implementace se v této variantě nevyžaduje; rozpory musí jít odhalit z pravidel a příkladů.

Teoretická cesta: ukázka požadovaného důkazu

Vlastní model: `var` zakládá místní proměnnou; přiřazení mění nejbližší existující proměnnou. D_0, D_1 označují deklarace.

```
var x = 2;
{
  var x = 5;
  print x;
}
print x;
```

Krok	Dostupné vazby a účinek
Vnější `var`	$x \to D_0$, hodnota 2.
Vnitřní `var`	$x \to D_1$, hodnota 5; zastíní D_0.
První tisk	Čte D_1, vypíše 5.
Konec bloku	D_1 přestane být dostupné.
Druhý tisk	Čte D_0, vypíše 2.

Jde o tělo programu modelového jazyka.

Změna při obhajobě: nahradíme vnitřní deklaraci `var x = 5;` přiřazením `x = 5;`. Proč se výsledek změní na `5, 5`?

Společné mezníky

Odevzdání	A: Mini	B: návrh bez implementace
Do bloku 03	Resolver, vlastní test stínění a odmítnutí špatného jména.	Účel, gramatika a pravidla jmen; první AST a stopa vazeb.
Do bloku 04	Generování příkazů; test smyčky a předčasného návratu.	Pravidla podmínek a smyček; úplná stopa alespoň jedné iterace a ukončení.
Týden před ústním zkoušením	Binární runtime operace, typové chyby a vysvětlení celého běhu.	Pravidla hodnot a operací; všech deset případů, sjednocená finální specifikace.

Při každém odevzdání: co se změnilo, proč je to správně, který nový případ to ukazuje a jaká pomoc byla použita.

Obhajoba: ověřuji porozumění každého studenta

Teoretická cesta:

1. Student vysvětlí jednu vlastní volbu a ukáže pravidlo nebo místo v kódu, které ji zajišťuje.
2. Předem předpoví výsledek nového příkladu vybraného vyučujícím, nebo chybu a okamžik jejího vzniku.
3. Na místě změní jedno pravidlo či část řešení a odvodí dopad na výstup a test.

Hodnotíme správnost pravidel, jejich soulad s příklady, zdůvodnění a přenos na nový případ. Samotný počet zelených testů nestačí.

Návrh režimu AI: při přípravě povolena; uvést nástroj, převzaté části a jejich ověření. Při individuálním ověření bez generování odpovědí. U týmové práce ověříme každého člena.